

Nothing is deployed. You are establishing who you are to AWS and GCP in a way that expires on its own, and putting a ceiling on what the rest of the curriculum can cost you.

THE HOPS

1

`browser console session` → `aws login`**a session reference — not a secret**

No `~/.aws/credentials` file is written. The CLI stores a pointer and refreshes it while the refresh token is valid.

2

`aws login session` → `credential_process`**materialized short-lived credentials, per invocation**

Terraform's Go SDK cannot read a session reference. The profile shells out to the CLI each time, so nothing is snapshotted into your shell.

3

`you` → `AWS Budgets`**a dollar threshold and an email address**

Set before the first apply. This is the only step here that has ever saved anyone real money.

WHERE IT COMES TO REST

`~/.aws/config`

A profile definition. No secret material.

READS You, and every tool honouring the AWS SDK contract.

AWS session store

The refreshable session behind `aws login`. Lapses on its own.

READS The CLI on this machine.

Terraform's record of what it built stops living on your laptop. From here on state is a versioned, encrypted object in S3, and a lock stops two applies from writing over each other.

THE HOPS

1

`terraform/bootstrap` → `AWS API``the backend itself: bucket, lock table, CMK`

This one workspace keeps local state, because it is the thing that creates the place state goes.

2

`terraform apply` → `DynamoDB``a lock item, held for the duration of the apply`

Released on completion. A crashed apply leaves it behind, which is what force-unlock is for.

3

`terraform apply` → `s3://state/labs/<lab>/terraform.tfstate``every resource, attribute and ID Terraform manages`

Including anything the provider marked sensitive. The encryption here is not decoration.

4

`backend.hcl` → `terraform init``bucket name and key ARN, supplied at init`

Both carry your account ID, and the capstone repo is public, so they are never written into the backend block.

WHERE IT COMES TO REST

`s3://state/.../terraform.tfstate`

Versioned, SSE-KMS. Prior versions survive a bad apply.

READS Anyone holding both `s3:GetObject` and `kms:Decrypt` on the CMK. State is plaintext to them.

`dynamodb tflock`

One row per in-flight apply.

READS Terraform, and you when an apply dies mid-flight.

You describe eleven controls as Terraform. AWS enforces them at request time — a non-TLS request is refused by the bucket, not by a policy document — and the resulting state file becomes the machine-readable proof.

THE HOPS

1

`main.tf` → `AWS API`

20 resources: 2 buckets, a CMK, and their configuration

Each control cited in the lab is enforced by a line you can point at. If you cannot point at the line, it does not go in the table.

2

`any client PUT` → `bucket policy evaluation`

allowed, or 403

Two denies: `aws:SecureTransport = false`, and an upload naming the wrong key. Encryption stops being a default and becomes a condition.

3

`S3 log delivery` → `log bucket access-logs/`

one line per request against the primary bucket

Granted by a bucket policy with confused-deputy conditions, not by re-enabling ACLs — which would itself be a Security Hub finding.

4

`terraform show -json` → `evidence/lab-2-3/state.json`

the applied configuration, as JSON

Lab 6.1 points SC-28's implemented-requirement at this file. An assessor follows the link and sees what you saw.

WHERE IT COMES TO REST

primary bucket

Your data. SSE-KMS with bucket keys, versioned, four public-access flags, ACLs off.

READS Principals with S3 access AND `kms:Decrypt`. The key is the real access control.

log bucket

Who touched the primary. Versioned, so an entry cannot be silently overwritten. 90-day lifecycle.

READS Nothing, yet. Collection without review is the gap the lab names out loud.

evidence/lab-2-3/state.json

In your repository, committed.

READS Anyone reading your OSCAL, by following the href.

The same argument on another cloud, expressed as a module. A consumer supplies the handful of values a business actually varies and cannot opt out of the controls, because the controls are not parameters.

THE HOPS

1

`consumers/dev/main.tf` → `module call``project, environment, retention, labels`

Everything compliance-relevant — encryption, uniform access, versioning, logging — is hardcoded inside and not exposed.

2

`module` → `GCP API``a key ring, a rotating CMEK, a bucket, and its log target`

Rotation is 90 days, which is where the per-version cost comes from.

3

`module outputs` → `the caller's output``a computed attestation`

A statement about what was enforced, derived from the resources themselves. Lab 6.1 maps it into OSCAL.

WHERE IT COMES TO REST

the GCS bucket

CMEK-encrypted with a key you rotate, uniform access, public access prevention enforced.

READS IAM only. Uniform access removes per-object ACLs as a path.

the log bucket

Named as the logging target, and versioned for the same reason as on AWS.

READS The bucket writer identity, plus whoever you grant.

A Terraform run becomes an object nobody can overwrite, including you. That object is what an auditor is handed instead of a screenshot.

THE HOPS

1

the Lab 2.3 workspace → capture-evidence.sh

tfplan, *.tf, module sources

Raw state is not captured by default. It carries sensitive attributes, and the bundle exists to be shared.

2

bundle.tar.gz → s3://VAULT/runs/RUN_ID/

the run, compressed

One prefix per run. The RUN_ID is how everything downstream refers to it.

3

S3 PUT → Object Lock

a retain-until date, applied at write

From here the object cannot be replaced or deleted, by any principal, until the date passes.

4

S3 response → you

a VersionId

The durable handle. Your OSCAL evidence links resolve by this, not by a path that could be repointed.

WHERE IT COMES TO REST

the evidence vault

Object Lock, versioning, SSE-KMS, four public-access flags, and a policy denying s3:DeleteBucket to everyone but the account root.

READS Whoever you grant — and from Lab 5.2, every read is a CloudTrail data event.

Nothing is applied. The plan is turned into JSON and the policies read that JSON, so the check happens before the resource exists rather than after it is found.

THE HOPS

1

`fixtures/terraform → terraform plan -out tfplan`

a binary plan, good buckets and bad ones

Each bad fixture is named for the way it is wrong: bad_no_cmek, bad_public, bad_log_target.

2

`terraform show -json → plan.json`

`resource_changes[].change.after`

That path is the shape a policy actually reads. Everything else in the plan is noise to the gate.

3

`plan.json + policies → conftest`

deny messages, or silence

Five controls: SC-28, AC-3, CM-6, AU-3, AU-9. Each has a test asserting both a pass and a fail.

4

`policies → opa inspect --annotations`

`evidence/lab-3-3/policy-catalog.json`

The control mapping, extracted from the metadata rather than retyped. Lab 6.1 reads this file.

WHERE IT COMES TO REST

policy-catalog.json

Five entries: control, policy file, description.

READS `gen-oscal-requirements.py` in Lab 6.1. It is the single source of truth for the mapping.

nothing in the cloud

No apply happens in this lab. The whole exercise is plan-time.

READS —

The same five controls against AWS plans, plus the gate script CI will call verbatim. Deliberately broken copies of Lab 2.3 prove the gate can actually fail.

THE HOPS

1

`lab-2-3/*.tf` → `broken/``copies, degraded on purpose`

A policy you have never seen fail is a policy you do not know works.

2

`plan.json` → `scripts/policy-gate.sh``a verdict, and a count of policies loaded`

Both matter. The count is what the guard checks.

3

`the guard` → `exit code``non-zero when zero policies loaded`

This is the exact script Lab 4.3 shells out to. CI runs your script, not a reimplementaion of it.

WHERE IT COMES TO REST

the gate output

Written to the evidence directory alongside the plan it judged.

READS The pipeline, and anyone reading the run.

The same scripts run on a machine you do not control. It proves its identity with a token GitHub mints for that run, so there is no key anywhere to steal.

THE HOPS

1

`GitHub Actions` → `GitHub's OIDC issuer``a short-lived JWT naming repo, ref and workflow`

Minted per run. It does not exist before the job starts or after it ends.

2

`the JWT` → `sts:AssumeRoleWithWebIdentity``temporary AWS credentials, minutes long`

The trust policy pins which repository and ref may assume the role.

3

`the runner` → `plan` → `policy-gate.sh``pass or fail, fail closed`

Every action pinned to a SHA, so the gate cannot silently start running different code.

4

`merge to main` → `apply` → `state backend``the same state keys your laptop writes`

This only works because Lab 2.2 exists. On local state a fresh runner starts empty and either collides or duplicates.

WHERE IT COMES TO REST

no credential at rest

Nothing in this flow is stored. The token expires, the credentials expire.

READS Only the run that minted it.

the state backend

Written by CI on merge, read by your laptop, protected by the same lock.

READS The OIDC role, scoped, plus you.

Object Lock proves the bundle did not change. A signature proves who deposited it. Verification needs both, and it has to care whose signature it is.

THE HOPS

1

`the bundle → cosign sign`**a signature and a certificate**

The identity in the certificate comes from the same OIDC token, so it names the workflow that produced the bundle.

2

`the signature → the vault`**stored beside the bundle, under retention**

Both objects are locked. Neither can be swapped for a friendlier version.

3

`an auditor → verify-evidence.sh RUN_ID`**CHAIN INTACT, or a non-zero exit**

Downloads, verifies the signature against an identity regex, and checks the bundle is complete.

WHERE IT COMES TO REST

vault objects + signatures

Retained together. The pair is the record; either alone proves less than you would want.

READS The grader, running your script against your vault.

Until now logs were collected. Here something reads them. That is the entire difference between AU-3 and AU-6, and Athena is what it costs.

THE HOPS

1

every AWS API call → CloudTrail

management events, plus data events scoped to the vault

Data events bill per event. Pointed at a busy bucket the number changes character, which is why the scope is one bucket.

2

CloudTrail → the trail bucket

gzipped JSON, on a lifecycle

Retention is a control and a budget. Unbounded logs are an unbounded bill.

3

the trail bucket → Glue catalog → Athena

a queryable table over that JSON

Three queries constitute the audit review. Saving them is what makes it repeatable rather than a one-off.

4

your resources → Security Hub

findings against NIST 800-53 and AWS FSBP

Continuous, and it will judge the buckets you built in Chapter 2.

WHERE IT COMES TO REST

the trail bucket

The record of who did what. Hardened like every other bucket in the curriculum.

READS Athena, and anyone with read on the bucket. Protect it like the vault.

evidence/lab-5-2/

The three queries and their results, committed.

READS Your OSCAL AU-6 statement points here.

Prevention instead of detection. An org policy makes a misconfiguration impossible to create, rather than something you find later and file a ticket about.

THE HOPS

1

any create request → Org Policy evaluation

allowed, or refused at the API

A bucket without uniform access cannot be created. There is no drift to detect because there is no drift.

2

GitHub Actions → Workload Identity Federation

a short-lived GCP token via a service account

No service account key is ever created. Disabling SA keys is itself one of the four org policies.

3

reads and writes → Data Access logs

per-service audit records: storage, kms, iam

Off by default on GCP, and enabled explicitly per service. That explicitness is what your AU-2 statement describes.

WHERE IT COMES TO REST

Cloud Logging

Data Access log entries for the three services you enabled.

READS Project viewers with the logging role, plus your queries.

org policy

Configuration, not data. It is enforcement, and it applies before anything is written.

READS Every create request in the project, whether or not it knows.

Everything above becomes one document with links in it. The document is only worth something if the links resolve, and OSCAL will not check that for you.

THE HOPS

1

`policy-catalog.json` → `gen-oscail-requirements.py``implemented-requirement` entries

Projected from Lab 3.3, not retyped. The mapping was authored once, in the policy that enforces it.

2

`requirements + module paths + VersionIds` →`component-definition.json``a description of the system you built`

Implementation status is implemented, partial, or inherited. A component claiming twelve where three are partial is the failure the brief names.

3

`the profile` → `NIST 800-53 catalog, pinned tag``the control IDs you actually claim`

Catalog, profile and component must share an oscal-version. Mixing them is the most common validation failure.

4

`component + profile` → `verify-oscail-graph.sh``OSCAL GRAPH INTACT, or BROKEN`

Fetches every evidence href. This is the step that turns a valid document into a true one.

WHERE IT COMES TO REST

`oscail/`

JSON in your repository. Nothing is created in the cloud by this lab.

READS An assessor, who follows the links from here into the vault.

`evidence/lab-6-1/`

The validate and graph-verify output, committed.

READS The grader reads these before reading the JSON.